STAJ PROJESİ · TÜRKSAT AVIATION AND COMMUNICATION

Akıllı Okul Bilgi Sistemi

Hafta 2 — FastAPI İskeleti ve Kimlik Doğrulama

HAZIRLAYAN	Bera Ergin — s29315
KURUM	PJAIT — 7. Yarıyıl
BU HAFTA	Hafta 1'in RLS garantisi, API katmanından geçen her isteğe taşınıyor

Nereden Geliyoruz, Nereye Gidiyoruz

HAFTA 1 — BİTTİ

- ✓ PostgreSQL + Row-Level Security kuruldu
- ✓ 3 test: teacher→4, principal→12, parent→1
- ✓ Hiçbir API, hiçbir kullanıcı girişi yok
- ✓ Yalnızca veritabanı, kendi başına doğru filtreliyor

Henüz kimse giriş yapamıyor, RLS bağlamını kim kuracak belli değil.

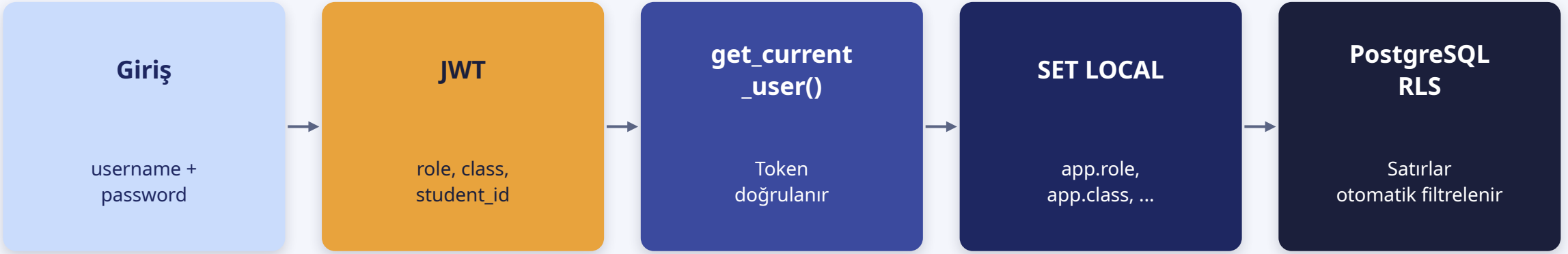
HAFTA 2 — HEDEF

- FastAPI + JWT ile giriş
- Giriş yapan kullanıcının rolü/sınıfı belirleniyor
- Bu bilgi, her istekte PostgreSQL oturumuna bildiriliyor
- Hafta 1'in RLS kanıtı artık API üzerinden de doğrulanıyor

Henüz LLM/sorgu akışı yok — yalnızca kimlik ve bağlam zinciri.

Bağlam Zinciri: Giriş'ten RLS'e

Hafta 1'de kurulan RLS'in, her API isteğinde otomatik devreye girmesini sağlayan zincir



Kritik tasarım kararı

FastAPI hiçbir zaman "bu kullanıcı şu satırları görebilir" diye bir karar vermiyor — yalnızca kimliği doğruluyor ve bu kimliği PostgreSQL oturumuna bildiriyor. Hangi satırların döneceğine, Hafta 1'de kurulan RLS politikaları karar veriyor. API katmanı, erişim kuralının kendisini hiç bilmiyor.

Kimlik Doğrulama Akışı

Kullanıcı adı/şifre → JWT token → korumalı uç noktalar



Örnek JWT payload

```
{ "sub": "teacher1", "role": "teacher", "related_class": "9-A", "exp": 1735729200 }
```

Şifre Güvenliği

Hafta 1'deki düz metin şifrelerden bcrypt hash'e geçiş

Hafta 1

```
password_hash = "teacher123"
```

Düz metin olarak saklanıyordu — WEEK_01.md'de bilinçli bir basitleştirme olarak belgelenmişti.

Hafta 2

```
password_hash = "$2b$12$KIXx..."
```

get_password_hash() ile bcrypt hash'i üretiliyor. Kolon adı aynı kalıyor — sadece içerik değişiyor, modelde değişiklik gerekmiyor.

Neden önemli

Veritabanı ele geçirilse bile şifreler doğrudan okunamaz. passlib[bcrypt] her şifre için ayrı bir salt üretir, aynı şifreye sahip iki kullanıcının hash'i bile farklı çıkar.

Rol Bazlı Giriş + RLS Doğrulaması

Her rol giriş yapıyor, aynı /debug/students uç noktasına istek atıyor — dönen satır sayısı Hafta 1'in sonuçlarıyla eşleşmeli

Rol	Kullanıcı Adı	Bağlam	Beklenen Sonuç
Müdür	`principal`	—	200 + 12 satır
Öğretmen	`teacher1`	9-A	200 + 4 satır
Veli	`parent1`	Ali Yılmaz	200 + 1 satır
Öğrenci	`student1`	Ali Yılmaz	200 + 1 satır
Geçersiz	`teacher1 / yanlış şifre`	—	**401 Unauthorized**

Bu tablo, Hafta 1'in test_rls.py sonuçlarının (4/12/1) artık API katmanından, gerçek kullanıcı girişiyle de aynen doğrulandığını gösteriyor.

Geçici Bir Kanıt Endpoint'i

GET /debug/students — bilerek geçici, bilerek belgelenmiş

Bu hafta henüz LLM/sorgu akışı yok — ama bağlam zincirinin (giriş → JWT → SET LOCAL → RLS) gerçekten çalıştığını bir şekilde kanıtlamamız gerekiyor. Bunun için yalnızca "SELECT id, name, class_name FROM students" çalıştıran, sabit, basit bir uç nokta eklendi.

Bu endpoint ayrı bir dosyada (backend/app/api/debug.py) tutuluyor ve Hafta 4'te LLM entegrasyonu gelince kaldırılacak — gerçek /query endpoint'i onun yerini alacak.

Neden ayrı bir dosyada

query.py içine karıştırılmıyor — böylece Hafta 4'te bu geçici kodu silmek, gerçek sorgu mantığına hiç dokunmadan tek bir dosyayı kaldırmak kadar basit olacak.

Hafta 2 — Kapsam ve Kabul Kriterleri

Bu hafta teslim edilenler

- ✓ FastAPI proje iskeleti (main.py, routers)
- ✓ security.py — JWT + bcrypt yardımcıları
- ✓ auth.py — POST /auth/login uç noktası
- ✓ dependencies.py — get_current_user() + RLS bağlamı
- ✓ Ayrı school_app bağlantısıyla SET LOCAL mekanizması
- ✓ Geçici /debug/students — RLS'in API'den kanıtı

Kabul kriterleri

1. uvicorn backend.app.main:app çalışır, /docs erişilebilir
2. 4 rolün her biri doğru şifreyle 200 + token döner
3. Yanlış şifre 401 Unauthorized döner
4. /debug/students, her rol için Hafta 1'deki sayılarla (4/12/1/1) eşleşir
5. school_app rolü superuser değildir — RLS gerçekten devrede

Sonraki Adım — Hafta 3

Kapsam yeniden değerlendiriliyor

Orijinal plan, Hafta 3'te bir RuleEngine sınıfı (rol bazlı SQL filtresi üreten Python kodu) inşa etmekti. Ama bu işi artık PostgreSQL'in kendisi, Hafta 1-2'de kurulan RLS ile veritabanı seviyesinde yapıyor. Hafta 3'ün kapsamı, bu gerçeğe göre yeniden belirlenecek.

Teşekkürler